

Multimodal Fashion & Context Retrieval

An intelligent search engine that retrieves images from a diverse fashion dataset given natural-language descriptions — understanding *what* someone is wearing, *where* they are, and the overall *vibe* of the attire.

Repository: <https://github.com/buddywhitman/multimodal-fashion-retrieval>

1. Approaches

Ways to build natural-language → image retrieval for fashion, and when each makes sense:

Approach	How it works	Good for	Weakness
Keyword / filename matching	Match query words against filenames or manual tags	Trivial to stand up	No generalization; can't handle synonyms or unseen phrasings
Trained multi-label classifier + filter	Train a CNN on a fixed category/attribute taxonomy, filter images by predicted labels	High precision on labels seen in training	No zero-shot ability — can't handle open vocabulary ("bright yellow raincoat") or scene/style/vibe language; needs retraining to add a concept
Vanilla CLIP, single global embedding	Encode whole image and whole query into one shared vector space, rank by cosine similarity	Genuinely zero-shot; strong on scene, style, and "vibe" language	Pooling a whole image into one vector loses compositional structure: "red tie with a white shirt" and "white tie with a red shirt" embed almost identically. Also fuzzy on fine-grained color
Region-based detection + captioning	Run/train an object detector, caption each region, embed each caption separately	Best possible fine-grained, compositional accuracy	Requires running or training a detector at index time; heavier pipeline and more failure surface than the problem warrants
Hybrid: fashion-domain CLIP (whole-image + per-garment region) + structured attributes from existing segmentation, re-ranked (chosen)	Whole-image CLIP for scene/vibe, plus per-garment crops and mask-derived (category, color) attributes for exact compositional matching	Zero-shot <i>and</i> compositional, using annotations the dataset already provides — no detector training, no new labeling	Structured signal is bounded by the category vocabulary and by what exists in the corpus; both measured explicitly

The core tension is **zero-shot generalization vs. compositional precision**. Vanilla CLIP has the former and lacks the latter; a trained classifier has the latter and lacks the former. The chosen approach keeps CLIP's zero-shot behavior as the fallback for everything, and layers exact, instance-scoped attribute matching on top for the compositional cases where CLIP alone fails.

2. Chosen Approach

2.1 Dataset

The corpus is Fashionpedia — its `val2020` split plus a sampled subset of the larger `train2020` split: **15,189 images, 110,907 annotated garment instances**. Both splits share Fashionpedia's instance-segmentation format (per-garment polygons + a 46-category taxonomy). `train2020` is included so the evaluation queries' attribute combinations (a yellow coat, a red tie) have real ground truth to retrieve; sampling takes every `train2020` image containing the rarer target categories plus a reproducible random sample of the rest for diversity. The corpus spans the three required axes — environment (office, street, park, home, studio, runway, beach), clothing type (formal, casual, outerwear, sporty), and a wide color palette.

2.2 Architecture

Part A — Indexer (`indexer/`)

1. `dataset.py` loads and merges the Fashionpedia annotations into plain per-image records; each garment instance carries its category label and segmentation mask.
2. `embed.py` embeds with `patrickjohncyh/fashion-clip` — CLIP fine-tuned on ~800K fashion image/text pairs, so it grounds garment types, colors, and styles far better than general-purpose CLIP. It runs on GPU when available and produces two representations per image: the **whole image** (scene, setting, overall style) and **each garment cropped from its bounding box, embedded separately** — the key to compositionality (§2.3).
3. `color_extract.py` + `common/colors.py` derive each garment's color(s) from its segmentation mask (mask eroded inward to avoid background bleed, median RGB over a cropped region), named by nearest-neighbor in **CIE Lab space** (perceptually uniform) against a reference palette built from the **XKCD color survey** (~800 human color-name judgments). Chromatic *families* (blue / navy / denim / cornflower blue) are derived programmatically so a query for "blue" gets full credit for an exact match and graded credit for a family member. A second color is recorded for genuinely multi-colored (patterned or color-blocked) garments.
4. `relations.py` + `depth_relations.py` record which garments are **layered** together (bounding-box overlap within a body region) and, via a monocular depth model run only on those candidate pairs, the **z-order** — which garment is worn *over* the other.
5. `scene_tag.py` zero-shot-tags each image's **scene** (place type), **style**, and **weather** by cosine-matching the image embedding against small natural-language prompt banks.
6. `build_index.py` stores whole-image embeddings + all structured metadata in one **Chroma** collection and per-garment crop embeddings in another. Chroma is a single `pip install`, persistent, with metadata filtering built in — the pragmatic vector-store choice so effort goes into ML logic, not storage engineering.

Part B — Retriever (`retriever/`)

1. `query_parser.py` turns a free-text query into structured signals: (category, color) garment pairs, scene/style/weather terms, and layering relations with direction ("jacket **over** shirt"). Multi-word color names ("marine blue") are matched as single tokens. Garment words outside the known vocabulary are resolved **zero-shot** (§2.4).
2. `search.py` combines whole-image CLIP similarity (Chroma HNSW) with deterministic exact recall of every image containing a garment of the requested (category, color-family) via an in-memory attribute index, then scores each candidate:

```
score = W_CLIP · image_similarity + W_COMP · attribute_overlap + W_SCENE ·  
(scene/style/weather match)
```

renormalized over whichever signals the query produced. `attribute_overlap` gives **1.0** for an exact per-garment color match, **0.8** for a same-family match, **0.6** for category-only, plus a bonus when a requested layering direction is confirmed by the stored z-order. Weights (`0.20 / 0.70 / 0.10`) are chosen by ablation (§3.2). If a query produces no structured signal, it falls back to pure CLIP similarity — the hybrid machinery only ever sharpens results, never blocks a zero-shot query.

2.3 How it handles fashion queries

Compositionality — the hard case ("red tie with a white shirt" vs. "white tie with a red shirt"). A single pooled image embedding contains "red", "white", "tie", "shirt" for *both* phrasings and cannot separate them. This system embeds each garment crop independently and re-ranks with **exact, instance-scoped** (`category, color`) attributes from the segmentation masks, so "tie = red" and "shirt = white" are matched to specific garments and cannot bleed into each other. A controlled color-swap experiment measures this directly: whole-image CLIP discriminates the correct composition from the swapped one at **0.60** accuracy (near chance), while the hybrid reaches **1.00** (mean margin +0.38).

Fine-grained color — Lab-space naming over an ~800-name palette distinguishes shades fashion queries depend on (burgundy vs. maroon, navy vs. denim, khaki vs. olive), and chromatic-family matching means a broad color word still recalls all its specific shades.

Context / "where" and "vibe" — the whole-image embedding plus zero-shot scene/style/weather tags handle "inside a modern office", "casual weekend outfit", "a rainy day". Adding a location or style term to a query measurably sharpens it (§3.3).

Layering — depth-verified z-order answers "a jacket over a shirt" as a directed relation, not merely co-occurrence.

2.4 Zero-shot capability

Two levels. First, the whole-image CLIP embedding handles descriptions that match no explicit label — "an elegant evening gown for a gala" surfaces dress + formal + runway images despite none of those words being a stored label. Second, the query parser is *itself* zero-shot for garment vocabulary: unknown words ("windbreaker", "parka", "loafers") are resolved through fashion-CLIP via a two-prototype classifier that accepts a word as a garment only if it is more garment-like than place/action/person/time-like, avoiding false matches on function words. This is calibrated to **8/8 recall, 3/3 precision** on a held-out test.

3. Benchmarking & Profiling

All results are reproducible from `eval/`. Ground truth is Fashionpedia's own segmentation + category labels read back from the built index, so relevance is judged against real corpus attributes, not hand-labeling.

3.1 The 5 evaluation queries

Precision@5, with color relevance judged by chromatic family to match how the retriever scores:

#	Query	P@5
1	"A person in a bright yellow raincoat."	1.00
2	"Professional business attire inside a modern office."	1.00
3	"Someone wearing a blue shirt sitting on a park bench."	1.00
4	"Casual weekend outfit for a city walk."	1.00
5	"A red tie and a white shirt in a formal setting."	1.00
	mean	1.00

3.2 Corpus-grounded benchmark

Five prompts are too small a sample to trust. A query is auto-generated for **every** (category, color) combination occurring ≥ 3 times in the corpus, checking whether the true-positive images land in the top 5:

Slice	# queries	mean P@5	mean R@5
GARMENT (wearable clothing types)	2,010	0.817	0.913
PART (hardware/embellishments — harder, out of scope per PRD)	1,630	0.793	0.870
ALL	3,640	0.806	0.894

P@5 is bounded by the metric itself: 34% of combinations have fewer than 5 true positives, so their P@5 caps below 1.0 mechanically (a combination with 3 true positives caps at $3/5 = 0.60$). The mean *achievable* P@5 ceiling is **0.888**; the GARMENT slice at 0.817 sits close to it, and R@5 (0.913) is near its own ceiling.

Weight ablation (GARMENT combos):

(w_CLIP, w_COMP, w_SCENE)	mean P@5	mean R@5
0.70 / 0.20 / 0.10 (CLIP-heavy)	0.433	0.485
0.35 / 0.55 / 0.10	0.785	0.889
0.20 / 0.70 / 0.10 (chosen)	0.805	0.916
0.10 / 0.80 / 0.10	0.805	0.916

Exact (category, color) matches are ground truth from segmentation and a more reliable ranking signal than fuzzy CLIP similarity once a candidate is known to contain the right garment and color, so the optimum is comp-

heavy. `w_CLIP = 0.20` is the plateau knee — it captures the full gain while keeping CLIP weight at twice the scene weight, a robustness margin for pure-scene queries (verified unchanged across the sweep).

3.3 Context awareness (color + type + location)

Building the literal three-attribute query for real `(category, color, scene)` combinations and comparing to the same query with the location term removed:

Query form	mean P@5
color + type only	0.29 – 0.38
color + type + location	0.59 – 0.69
effect of the location term	+0.30

The location term roughly **doubles precision** — a given color+garment is often shared across many scenes, and the scene score resolves the ambiguity. The effect reproduces consistently across independent samples.

3.4 Backbone selection

The chosen `patrickjohncyh/fashion-clip` is compared against the larger `Marqo/marqo-fashionCLIP` on the compositional-discrimination task:

Backbone	Discrimination accuracy	Mean margin
patrickjohncyh/fashion-clip (chosen)	0.650	+0.006
Marqo/marqo-fashionCLIP	0.633	+0.006

The larger checkpoint does not outperform the chosen one on this task, so the smaller, faster model is used. The embedding module supports both HuggingFace-native and OpenCLIP-native checkpoints, so swapping backbones is a one-line config change.

3.5 Latency profile

Steady-state on the 15,189-image / 99,300-crop corpus:

Query type	median latency
single garment + color	~32 ms
two garments + style	~33 ms
scene/style only	~30 ms
high-recall (large color family)	~34 ms

Query cost is dominated by the fashion-CLIP text encode (15=20 ms). Exact recall and candidate scoring run against ~~in-memory indexes~~ `(category, color) → image ids`, and cached image metadata + embeddings) built once ~~at startup~~ (6 s, tens of MB), turning per-query attribute lookup and scoring into dict/numpy operations rather than repeated database scans.

3.6 Scalability to ~1M images

Per-query cost is flat with corpus size: the image ANN pulls a fixed-size candidate pool (Chroma HNSW, sub-linear), and exact recall + scoring are dict/numpy lookups. The in-memory indexes trade startup memory for query speed — negligible at 15K images; at 1M the image-embedding store (2 GB) would instead be served by Chroma's server/sharded mode, which the retriever isolates behind the standard collection API plus two swappable in-memory helpers. Indexing throughput is GPU-accelerated, with color extraction operating on garment crops rather than full images.

4. Codebase (GitHub) Link

<https://github.com/buddywhitman/multimodal-fashion-retrieval>

Organized as:

- `common/` — shared configuration and the color classifier (constants both pipelines depend on)
- `indexer/` — Part A: feature extraction and vector storage
- `retriever/` — Part B: query parsing, search logic, and in-memory indexes
- `eval/` — coverage checks, benchmarks, ablations, and regression tests, all independently reproducible
- `docs/` — this write-up and the original PRD

Setup, dataset-fetch, indexing, query, and evaluation instructions are in `README.md` .

5. Approaches for Future Work

5.1 Adding locations (cities, places) and weather

Place-type and weather are already inferred zero-shot — place *type* (office, park, beach, cafe, gym) rather than city names, because a street-style photo does not reliably reveal which city it was taken in. Extensions:

- **Finer place granularity** (mall, restaurant patio, subway platform): add prompts to the banks in `common/config.py` — the tagging and scoring machinery generalizes to any additional axis with no architecture change.
- **Real deployment metadata**: where EXIF, GPS, or capture-timestamp data exists, prefer it over pixel inference — geolocation gives true city/place, and timestamp + location joined against a weather API gives true weather. This turns "locations and weather" into a metadata-join problem, strictly more reliable, and slots into the same `(image → tag)` metadata fields the retriever already scores against.
- **A dedicated scene classifier** (e.g. Places365) as an alternative or complement to CLIP-prompt tagging for place type, if higher scene accuracy is needed.

5.2 Improving precision

- **A learned color namer** trained on labeled color data, replacing nearest-neighbor over a fixed palette — the Lab-space infrastructure is the right foundation; only the reference points would become learned.
- **A learned re-ranker** over the three scoring signals (image similarity, attribute overlap, scene/style/weather), replacing fixed weights with a model trained on relevance-labeled queries — capturing interactions a single global weight vector cannot.
- **Full z-order and true "tucked in"** reasoning beyond pairwise over/under, which requires occlusion signal (instance-segmentation overlap direction or a trained relation classifier) rather than relative depth alone.
- **A stronger or fashion-specialized backbone** as such checkpoints appear — the embedding module already supports both HuggingFace- and OpenCLIP-native formats, so evaluation and adoption are a config change plus a benchmark run.
- **Corpus growth** for the rarest attribute combinations — long-tail retrieval quality is ultimately bounded by how many true positives exist to retrieve, which more source data addresses directly.